COURSE TITLE: **SOFTWARE DEVELOPMENT AND MANAGEMENT**

PAPER CODE: **CSE 4401**

LECTURER: **MASESE CHUMA**

CONTACT: **0725999196**

Introduction

The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view.

Requirement Engineering

The process to gather the software requirements from client, analyze and document them is known as requirement engineering.

The goal of requirement engineering is to develop and maintain sophisticated and descriptive 'System Requirements Specification' document.

Requirement Engineering Process

It is a four-step process, which includes –

- Feasibility Study
- Requirement Gathering
- Software Requirement Specification
- Software Requirement Validation

Feasibility study

When the client approaches the organization for getting the desired product developed, it comes up with rough idea about what all functions the software must perform and which all features are expected from the software.

Referencing to this information, the analysts does a detailed study about whether the desired system and its functionality are feasible to develop.

This feasibility study is focused towards goal of the organization. This study analyzes whether the software product can be practically materialized in terms of implementation, contribution of project to organization, cost constraints and as per values and objectives of the organization. It explores technical aspects of the project and product such as usability, maintainability, productivity and integration ability.

The output of this phase should be a feasibility study report that should contain adequate comments and recommendations for management about whether or not the project should be undertaken.

Feasibility Study

1. For all new systems, the requirements engineering process should start with a feasibility study.
2. The input to the feasibility study is a set of preliminary business requirements, an outline description of the system and how thesystem is intended to support business processes.
3. The results of the feasibility study should be a report that recommends whether or not it is worth carrying on with the requirementsengineering and system development process.
4. A feasibility study is a short, focused study that aims to answer a number of questions:
    a) Does the system contribute to the overall objectives of the organization?
    b) Can the system be implemented using current technology and within cost and schedule constraints?
    c) Can the system be integrated with other systems which are already in place?
5. The issue of whether or not the system contributes to business objectives is critical. If a system does not support these objectives, ithas no real value to the business.
6. Carrying out a feasibility study involves information assessment, information collection and report writing.
7. The information assessment phase identifies the information that is required to answer these three questions. Once the information has been identified, you should talk with information sources to discover the answers to these questions. Some of the possible questionsthat may be put are:
    a) How would the organization cope if this system were not implemented?
    b) What are the problems with current processes and how would a new system help to solve these problems?
    c) What direct contribution will the system make to the business objectives and requirements?
    d) Can information be transferred to and from other organizational systems?
    e) Does the system require technology that has not previously been used in the organization?
    f) What must be supported by the system and what need not be supported?
8. In a feasibility study, you may consult information sources such as the managers of the departments where the system will be used,software engineers who are familiar with the type of the system that is proposed, technology experts and end-users of the system.
9. Once you have the information, you write the feasibility study report. You should make a recommendation about whether or not thesystem development should continue. In the report, you may propose changes to the scope, budget and schedule of the system and suggest further high-level requirements for the system.

Requirement Gathering

If the feasibility report is positive towards undertaking the project, next phase starts with gathering requirements from the user. Analysts and engineers communicate with the client and end-users to know their ideas on what the software should provide and which features they want the software to include.

Software Requirement Specification

SRS is a document created by system analyst after the requirements are collected from various stakeholders.

SRS defines how the intended software will interact with hardware, external interfaces, speed of operation, response time of system, portability of software across various platforms, maintainability, speed of recovery after crashing, Security, Quality, Limitations etc.

The requirements received from client are written in natural language. It is the responsibility of system analyst to document the requirements in technical language so that they can be comprehended and useful by the software development team.

SRS should come up with following features:

- User Requirements are expressed in natural language.
- Technical requirements are expressed in structured language, which is used inside the organization.
- Design description should be written in Pseudo code.
- Format of Forms and GUI screen prints.
- Conditional and mathematical notations for DFDs etc.

Software Requirement Validation

After requirement specifications are developed, the requirements mentioned in this document are validated. User might ask for illegal, impractical solution or experts may interpret the requirements incorrectly. This results in huge increase in cost if not nipped in the bud. Requirements can be checked against following conditions -

- If they can be practically implemented
- If they are valid and as per functionality and domain of software
- If there are any ambiguities
- If they are complete
- If they can be demonstrated

Requirement Elicitation Process

Requirement elicitation process can be depicted using the following diagram:

- **Requirements gathering -** The developers discuss with the client and end users and know their expectations from the software.
- **Organizing Requirements -** The developers prioritize and arrange the requirements in order of importance, urgency and convenience.
- **Negotiation & discussion -** If requirements are ambiguous or there are some conflicts in requirements of various stakeholders, if they are, it is then negotiated and discussed with stakeholders. Requirements may then be prioritized and reasonably compromised.
  The requirements come from various stakeholders. To remove the ambiguity and conflicts, they are discussed for clarity and correctness. Unrealistic requirements are compromised reasonably.
- **Documentation -** All formal & informal, functional and non-functional requirements are documented and made available for next phase processing.

Requirement Elicitation Techniques

Requirements Elicitation is the process to find out the requirements for an intended software system by communicating with client, end users, system users and others who have a stake in the software system development.

There are various ways to discover requirements

Interviews

Interviews are strong medium to collect requirements. Organization may conduct several types of interviews such as:

- Structured (closed) interviews, where every single information to gather is decided in advance, they follow pattern and matter of discussion firmly.
- Non-structured (open) interviews, where information to gather is not decided in advance, more flexible and less biased.
- Oral interviews
- Written interviews
- One-to-one interviews which are held between two persons across the table.
- Group interviews which are held between groups of participants. They help to uncover any missing requirement as numerous people are involved.

Surveys

Organization may conduct surveys among various stakeholders by querying about their expectation and requirements from the upcoming system.

Questionnaires

A document with pre-defined set of objective questions and respective options is handed over to all stakeholders to answer, which are collected and compiled.

A shortcoming of this technique is, if an option for some issue is not mentioned in the questionnaire, the issue might be left unattended.

Task analysis

Team of engineers and developers may analyze the operation for which the new system is required. If the client already has some software to perform certain operation, it is studied and requirements of proposed system are collected.

Domain Analysis

Every software falls into some domain category. The expert people in the domain can be a great help to analyze general and specific requirements.

Brainstorming

An informal debate is held among various stakeholders and all their inputs are recorded for further requirements analysis.

Prototyping

Prototyping is building user interface without adding detail functionality for user to interpret the features of intended software product. It helps giving better idea of requirements. If there is no software installed at client's end for developer's reference and the client is not aware of its own requirements, the developer creates a prototype based on initially mentioned requirements. The prototype is shown to the client and the feedback is noted. The client feedback serves as an input for requirement gathering.

Observation

Team of experts visit the client's organization or workplace. They observe the actual working of the existing installed systems. They observe the workflow at client's end and how execution problems are dealt. The team itself draws some conclusions which aid to form requirements expected from the software.

Software Requirements Characteristics

Gathering software requirements is the foundation of the entire software development project. Hence they must be clear, correct and well-defined.

A complete Software Requirement Specifications must be:

- Clear
- Correct
- Consistent
- Coherent
- Comprehensible

- Modifiable
- Verifiable
- Prioritized
- Unambiguous
- Traceable
- Credible source

Software Requirements

We should try to understand what sort of requirements may arise in the requirement elicitation phase and what kinds of requirements are expected from the software system.

Functional and Non-Functional Requirements

Software system requirements are often classified as functional, non-functional or domain requirements.

Functional Requirements

Requirements, which are related to functional aspect of software fall into this category.

They define functions and functionality within and from the software system.

1. These are statement of services the system should provide, how the system should react to particular inputs and how the systemshould behave in particular situations.
2. In some cases, the functional requirements may also explicitly state what the system should not do.
3. These requirements depend on the type of software being developed, the expected users of the software and the general approachtaken by the organization when writing requirements.
4. When expressed as user requirements, the requirements are usually described in a fairly abstract way.
5. Functional system requirements describe the system in detail, its inputs and outputs, exceptions, and so on.
6. In principle, the functional requirements specification of a system should be both complete and consistent. Completeness means thatall services required by the user should be defined. Consistency means that requirements should not have contradictory definitions.

For example, here are examples of functional requirements for a university library system called LIBSYS, used by the students and faculty toorder books and documents from other libraries.

a) The user shall be able to search either all of the initial set of databases or select a subset from it.

b) The system shall provide appropriate viewers for the user to read documents in the document store.

c) Every order shall be allocated a unique identifier (ORDER_ID), which the user shall

be copy to the document's permanentstorage area.

Examples -

- Search option given to user to search from various invoices.
- User should be able to mail any report to management.
- Users can be divided into groups and groups can be given separate rights.
- Should comply business rules and administrative functions.
- Software is developed keeping downward compatibility intact.

Non-Functional Requirements

Non Functional Requirements

1. These are constraints on the services or functions offered by the system.
2. They are not directly concerned with the specific functions delivered by the system.
3. They include timing constraints, constraints on the development process and standards.
4. They may relate to emergent system properties such as reliability, response time and store occupancy. Alternatively, they may define constraints on the system such as the capabilities of I/O devices and the data representations used in system interfaces.
5. Non-functional requirements often apply to the system as a whole. They do not usually just apply to individual system features orservices.
6. Failing to meet a non-functional requirement can mean that the whole system is unusable.

   For e.g., if an aircraft system does not meet its reliability requirements, it will not be certified as safe for operation; if a real-time controlsystem fails to meet its performance requirements, the control functions will not operate correctly.
7. Non-functional requirements arise through user needs, because of budget constraints, because of organizational policies, because of the need for interoperability with other software or hardware systems, or because of external factors such as safety regulations or privacylegislation.

Requirements, which are not related to functional aspect of software, fall into this category. They are implicit or expected characteristics of software, which users make assumption of.

Non-functional requirements include -

- Security
- Logging
- Storage
- Configuration
- Performance
- Cost
- Interoperability
- Flexibility
- Disaster recovery
- Accessibility

Requirements are categorized logically as

- **Must Have** : Software cannot be said operational without them.
- **Should have** : Enhancing the functionality of software.
- **Could have** : Software can still properly function with these requirements.
- **Wish list** : These requirements do not map to any objectives of software.

While developing software, 'Must have' must be implemented, 'Should have' is a matter of debate with stakeholders and negation, whereas 'could have' and 'wish list' can be kept for software updates.

User Interface requirements

UI is an important part of any software or hardware or hybrid system. A software is widely accepted if it is -

- easy to operate
- quick in response
- effectively handling operational errors
- providing simple yet consistent user interface

User acceptance majorly depends upon how user can use the software. UI is the only way for users to perceive the system. A well performing software system must also be equipped with attractive, clear, consistent and responsive user interface. Otherwise the functionalities of software system can not be used in convenient way. A system is said be good if it provides means to use it efficiently. User interface requirements are briefly mentioned below -

- Content presentation
- Easy Navigation
- Simple interface
- Responsive
- Consistent UI elements
- Feedback mechanism
- Default settings
- Purposeful layout
- Strategical use of color and texture.
- Provide help information
- User centric approach
- Group based view settings.

Software System Analyst

System analyst in an IT organization is a person, who analyzes the requirement of proposed system and ensures that requirements are conceived and documented properly & correctly. Role of an analyst starts

during Software Analysis Phase of SDLC. It is the responsibility of analyst to make sure that the developed software meets the requirements of the client.

System Analysts have the following responsibilities:

- Analyzing and understanding requirements of intended software
- Understanding how the project will contribute in the organization objectives
- Identify sources of requirement
- Validation of requirement
- Develop and implement requirement management plan
- Documentation of business, technical, process and product requirements
- Coordination with clients to prioritize requirements and remove and ambiguity
- Finalizing acceptance criteria with client and other stakeholders

Software Metrics and Measures

Software Measures can be understood as a process of quantifying and symbolizing various attributes and aspects of software.

Software Metrics provide measures for various aspects of software process and software product.

Software measures are fundamental requirement of software engineering. They not only help to control the software development process but also aid to keep quality of ultimate product excellent.

According to Tom DeMarco, a (Software Engineer), "You cannot control what you cannot measure." By his saying, it is very clear how important software measures are.

Let us see some software metrics:

- **Size Metrics -** LOC (Lines of Code), mostly calculated in thousands of delivered source code lines, denoted as KLOC.
  Function Point Count is measure of the functionality provided by the software. Function Point count defines the size of functional aspect of software.
- **Complexity Metrics -** McCabe's Cyclomatic complexity quantifies the upper bound of the number of independent paths in a program, which is perceived as complexity of the program or its modules. It is represented in terms of graph theory concepts by using control flow graph.
- **Quality Metrics -** Defects, their types and causes, consequence, intensity of severity and their implications define the quality of product.
  The number of defects found in development process and number of defects reported by the client after the product is installed or delivered at client-end, define quality of product.
- **Process Metrics -** In various phases of SDLC, the methods and tools used, the company standards and the performance of development are software process metrics.
- **Resource Metrics -** Effort, time and various resources used, represents metrics for resource measurement.

Requirement Engineering Process

1. The goal of the requirement engineering process is to create and maintain a system requirement specification (SRS) document.
2. The overall process includes four high-level requirements engineering sub-processes.
3. These are concerned with
   a) Assessing whether the system is useful to the business i.e. feasibility study,
   b) Discovering requirements i.e. requirement elicitation and analysis,
   c) Converting these requirements into some standard form i.e. requirements specification, and
   d) Checking that the requirements actually define the system that the customer wants i.e. requirement validation.
4. Fig below illustrates the relationship between these activities. It also shows the documents produced at each stage of therequirements engineering process.

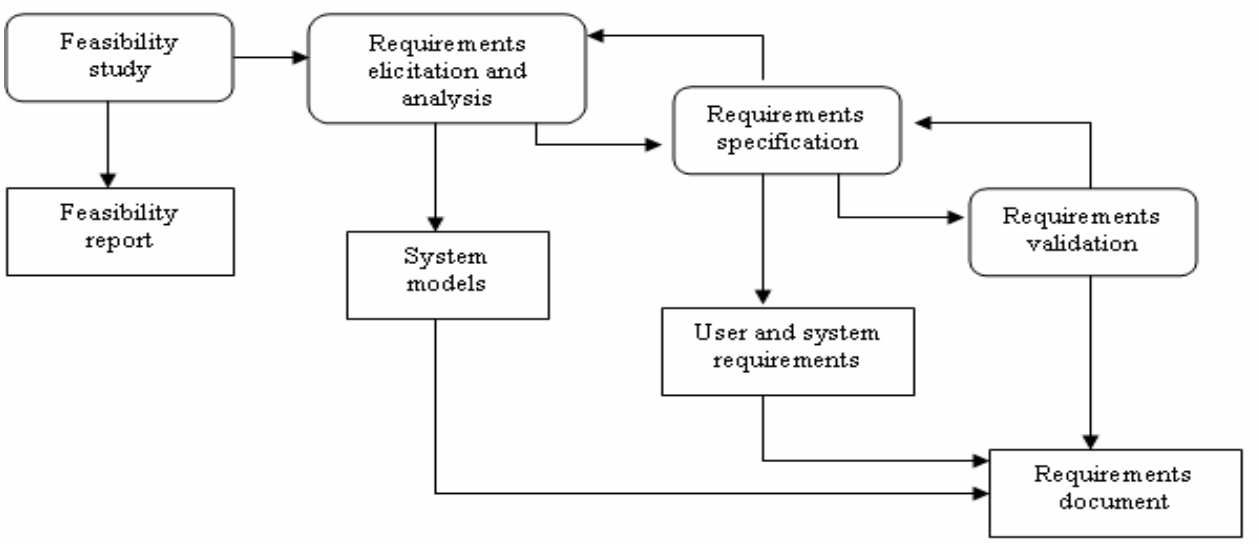

Fig Requirements engineering process

Note

Benefits of a Great SRS (Need for SRS)

The IEEE 830 standard defines the benefits of a good SRS:

1. Establish the basis for agreement between the customers and the suppliers on what the software product is to do. The complete description of the functions to be performed by the software specified in the SRS will assist the potential users to determine if the software specified meets their needs or how the software must be modified to meet their needs. [NOTE: Weuse it as the basis of our contract with our clients all the time].

2. Reduce the development effort. The preparation of the SRS forces the various concerned groups in the customer's organizationto consider rigorously all of the requirements before design begins and reduces later redesign, recoding, and retesting. Careful review of the requirements in the SRS can reveal omissions, misunderstandings, and inconsistencies early in the development cycle when these problems are easier to correct.
3. Provide a basis for estimating costs and schedules. The description of the product to be developed as given in the SRS is a realistic basis for estimating project costs and can be used to obtain approval for bids or price estimates. [NOTE: Again, we usethe SRS as the basis for our fixed price estimates]
4. Provide a baseline for validation and verification. Organizations can develop their validation and Verification plans muchmore productively from a good SRS. As a part of the development contract, the SRS provides a baseline against which compliance can be measured. [NOTE: We use the SRS to create the Test Plan].
5. Facilitate transfer. The SRS makes it easier to transfer the software product to new users or new machines. Customers thusfind it easier to transfer the software to other parts of their organization, and suppliers find it easier to transfer it to new customers.
6. Serve as a basis for enhancement. Because the SRS discusses the product but not the project that developed it, the SRS serves as a basis for later enhancement of the finished product. The SRS may need to be altered, but it does provide a foundation for continued production evaluation. [NOTE: This is often a major pitfall – when the SRS is not continually updated with changes]

<u>Characteristics of a great SRS</u>

An SRS should be

a) Correct – An SRS is correct if every requirements included in the SRS represents something required in the final system.

b) Unambiguous – An SRS is unambiguous if, and only if, every requirement stated therein has only one interpretation. Requirements are often written in natural language, which are inherently ambiguous. If the requirements are specified in a naturallanguage, the SRS write has to be especially careful to ensure that there are no ambiguities.

c) Complete – An SRS is complete if everything the software is supposed to do and the responses of the software to all classes ofinput data are specified in the SRS.

d) Consistent – An SRS is consistent if there is no requirement that conflicts with another. The SRS should be consistent withinitself and consistent to its reference documents. If you call an input "Start and Stop" in one place, don't call it "Start/Stop" in another.

e) Ranked for Importance – Generally all the requirements are not of equal importance. Some are critical, others are important but not critical and there are some which are desirable but not very important. Some requirements are core requirements which are notlikely to change as time passes, while others are more dependent on time. An SRS should be ranked for

importance and / or stability. Stability of a requirement reflects the chances of it changing in future.

f) Verifiable – An SRS is verifiable if and only if every stated requirement is verifiable. A requirement is verifiable if there existssome cost-effective process that can check whether the final software meets that requirement. This implies that the requirement

should have as little subjectivity as possible because subjective requirements are difficult to verify. Don't put in requirements like

- "It should provide the user a fast response." Another of my favorites is - "The system should never crash." Instead, provide aquantitative requirement like: "Every key stroke should provide a user response within 100 milliseconds."

g) Modifiable – An SRS is modifiable if its structure and style should be such that any necessary changes can be made easily whilepreserving completeness and consistency. Presence of redundancy is a major obstacle to modifiability, as it can easily lead to errors. For example, assume that a requirement is stated in two places and that the requirement later needs to be changed. If onlyone occurrence of the requirements is modified, the resulting SRS will be inconsistent.

h) Traceable – An SRS is traceable if the origin of each or its requirements is clear and if it facilitates the referencing of each requirement in future development. Forward traceability means that each requirement should be traceable to some design andcode elements. Backward traceability requires that it be possible to trace design and code elements to the requirements they support. Traceability aids verification and validation.